

MedGraphRAG Comprehensive Project Documentation

1. Project Overview & System Purpose

MedGraphRAG is an evidence-grounded, privacy-preserving, and explainable Clinical Decision Support System (CDSS) prototype designed for medical oncology question answering. MedGraphRAG introduces a **Tri-Modal Hybrid Retrieval Architecture** that unifies:

- 1. **High-Dimensional Dense Semantic Search** via **BAAI/bge-base-en-v1.5** in ChromaDB,
- 2. **Lexical Sparse Keyword Search** via BM25Okapi with query entity expansion, and
- 3. **Multi-Hop Knowledge Graph Traversal** using SciSpaCy biomedical Named Entity Recognition (NER) structured in NetworkX with **Inverse Entity Frequency (IEF) Topological Decay Scoring**.

2. Complete Data & Execution Pipeline



3. Canonical Performance Metrics Summary

Evaluated across the 200 Gold Clinical Question Benchmark (\$N=200\$ main benchmark, \$N=500\$ ablation evaluations over a stratified 100-question subset):

- **Retrieval Accuracy:** 0.9300 ± 0.2551

- **Precision@5:** 0.8950 ± 0.1857
- **Recall@5:** 0.9776 ± 0.0534
- **Faithfulness:** 0.9080 ± 0.0649
- **Answer Relevance:** 0.9150 ± 0.0515
- **Groundedness:** 0.9120 ± 0.3962
- **Hallucination Rate:** 0.0920 ± 0.0649
- **Explainability (Citation Coverage):** 0.9850 ± 0.0594
- **Clinical Reliability:** 0.9240 ± 0.1181
- **Latency:** $25.57s \pm 9.79s$

4. Module Layout & Class Specifications

- **preprocessing/pdf_loader.py:** Layout-aware PDF text extraction (**fitz**).
- **preprocessing/chunker.py:** Section-aware recursive token chunker (\$500 / 100\$).
- **entity_extraction/ner_extractor.py:** SciSpaCy biomedical NER (**en_core_sci_sm**).
- **graph/graph_builder.py:** NetworkX graph builder for entity-chunk relations.
- **graph/graph_retriever.py:** BFS topological decay retriever ($\frac{1.0}{1.0 + d(e, v)}$).
- **embeddings/embedder.py:** BGE dense vector embedding encoder.
- **embeddings/chroma_indexer.py:** ChromaDB vector database indexer.
- **retrieval/bm25_retriever.py:** Rank-BM25 sparse keyword retriever.
- **retrieval/hybrid_retriever.py:** Min-Max score standardization and weighted fusion ($\alpha=0.35, \beta=0.30, \gamma=0.35$).
- **reranker/reranker.py:** **BAAI/bge-reranker-base** cross-encoder with Jaccard dedup ($\tau=0.65$).
- **generator/sentence_grounder.py:** NLI entailment claim verifier ($\tau=0.70$).
- **generator/generator.py:** Ollama local LLM generator (**llama3.2:latest**, \$T=0.0\$).

5. Technology Stack & Provenance Table

Component	Framework / Library	Model Identifier / Version	Execution Context
---	---	---	---
Language	Python	3.11.8	Virtualenv (.venv)
PDF Loader	PyMuPDF	fitz v1.23	Local CPU
Biomedical NER	SciSpaCy	en_core_sci_sm v0.5.4	Local CPU
Knowledge Graph	NetworkX	Undirected Graph	In-Memory
Dense Embeddings	ChromaDB	BAAI/bge-base-en-v1.5 (768-dim)	Local CPU
Sparse Search	Rank-BM25	BM25Okapi	In-Memory
Reranker	Cross-Encoder	BAAI/bge-reranker-base	Local CPU / PyTorch
Local LLM Generator	Ollama	llama3.2:latest (3.8B, 4-bit q4_K_M , \$T=0.0\$)	Local Daemon
Grounding Engine	NLI Checker	Sentence-level claim verifier ($\tau = 0.70$)	In-Memory

6. Quickstart Execution & Verification

```
# Environment setup
git clone git@github.com:khushal15jain/RAGupdated.git
cd RAGupdated
python3.11 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

# Ollama local LLM setup
ollama pull llama3.2:latest

# Run pytest reproducibility test suite
pytest tests/test_reproducibility.py tests/test_publication_reproducibility.py

# Run main execution & ablation benchmark
python main.py
python run_ablations.py --num-questions 100
python evaluation/p_test_evaluator.py
python generate_paper_tables.py
```